

3) Open addressing

→ probe seq. $h(k, i)$ $i = \overline{0, m-1}$

Linear probing: $h(k, i) = (h'(k) + i) \% m$

$$h'(k) = k \% m$$

$$m = 16$$

Insert: 76, 12, 109, 43, 22, 18, 55, 81, 91, 27, 13, 16, 39

$$h(76, 0) = \cancel{76 \% 16} (76 \% 16 + 0) \% 16 = 12 \checkmark$$

$$h(12, 0) = (12 \% 16 + 0) \% 16 = 12 \rightarrow h(12, 1) = (12 \% 16 + 1) \% 16 = 13 \checkmark$$

$$h(109, 0) = 13 \rightarrow h(109, 1) = (109 \% 16 + 1) \% 16 = 14 \checkmark$$

$$h(43, 0) = 11 \rightarrow h(43, 1) = 11 \checkmark$$

$$h(22, 0) = 6 \checkmark$$

$$h(18, 0) = 2 \checkmark$$

$$h(55, 0) = 7 \checkmark$$

$$h(81, 0) = 1 \rightarrow h(81, 1) =$$

$$h(91, 0) = 11$$

$$h(27, 0) = 11$$

$$h(13, 0) = 13$$

$$h(16, 0) = 0$$

$$h(39, 0) = 7$$

$m = 15$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
27	81	18	13	16		22	55	39			43	76	12	109	91

(Note: In the original image, indices 8, 12, and 13 are circled with arrows pointing to 81, 76, and 12 respectively. The end of the array is crossed out.)

Quadratic probing

Double hashing

Insert, Search, remove

Lecher II.

1) Cuckoo hashing

T_1

T_2

add(e_i)

compute

position

→ if EMPTY $\Rightarrow$ place in T_1

if NOT EMPTY $\Rightarrow$ kick out (e_k) and put (e_i) there.

if empty $\checkmark$
if not

↳ kick out (e_k) and put (e_i) . . .

$m=11$.

$$h_1(k) = k \% 11$$

$$h_2(k) = (k / 11) \% 11$$

T_1

pos	0	1	2	3	4	5	6	7	8	9	10
T		100		36			50			20	
		67					105			53	
										75	53

T_2

pos	0	1	2	3	4	5	6	7	8	9	10
T	3	20			53			75		100	
					50						

1) insert key 20.

$$h_1(20) = 20 \% 11 = 9 \checkmark$$

4) 75

$h_1(75) = 9$ occupied $\Rightarrow$ put it anyway

$\Rightarrow$ take 53 $\Rightarrow h_2(53) = 4 \checkmark$

2) insert key 50.

$$h_1(50) = 6 \checkmark$$

5) 100

$$h_1(100) = 1 \checkmark$$

3) 53

$h_1(53) = 9$ occupied $\Rightarrow$ put it anyway

$\Rightarrow$ take 20 $\Rightarrow h_2(20) = 1 \checkmark$

6) 67

$h_1(67) = 1$ occupied $\Rightarrow$ put it anyway

$\Rightarrow$ take 100 $\Rightarrow h_2(100) = 9 \checkmark$

7) 105

$h_1(105) = 6$ occ. $\Rightarrow$ put it anyway.

$\Rightarrow$ take 50 $\Rightarrow h_2(50) = 4$ occupied $\Rightarrow$ put it anyway

$\Rightarrow$ take 53 $\Rightarrow h_1(53) = 9$

occupied $\Rightarrow$ put it anyway

$\Rightarrow$ take 75 $\Rightarrow h_2(75) = 6 \checkmark$

...more insertions...

8) 39. ...

2) Perfect hashing = hash table of hash tables

$\rightarrow$ no collisions

$\rightarrow$ know all elems in advance

$$M = m^2$$

Ex: "PERFECT HASHING EXAMPLE"

No collisions $\Rightarrow$ unique letters:

"PERFECTHASHINGXHL" $\Rightarrow N=15$.

we insert $N=15$ elems $\Rightarrow$ take $m=15$.

Letter	P	E	R	F	C	T	H	A	S	I	N	G	X	H	L
HashCode	16	5	18	6	3	20	8	1	19	9	14	7	24	13	12

index in the alphabet.

p = prime nr. larger than max key

$$\text{max key} = 24 \Rightarrow p = 29$$

$$\Rightarrow H_{a,b}(x) = ((a \cdot x + b) \% p) \% m ; a, b \text{ random}$$

$H(\text{HashCode})$	6	2	12	5	11	4	11	5	1	0	0	8	1	12	9
----------------------	---	---	----	---	----	---	----	---	---	---	---	---	---	----	---

$$H_{2,3}(16) = ((2 \cdot 16 + 3) \% 29) \% 15 = 6$$

$\text{pos} 0 \leftarrow I, N$

$I, N \Rightarrow \text{pos } 0 \Rightarrow$ secondary table with 4 pos. and $\neq$ hash functions, $(a, b:$

$S, X \Rightarrow \text{pos } 1 \Rightarrow$

$\hookrightarrow \text{ex: } a=4, b=11 \Rightarrow h(I)=h(9)=2;$

$h(N)=h(14)=1.$

$E \Rightarrow \text{pos } 2 \Rightarrow$ no collision $\Rightarrow$ secondary hash table with 1 elem, $h(x)=0$

$a=5, b=2.$

$h(S) = h(19) = ((5 \times 19 + 2) \% 29) \% 4 = 2$

$h(X) = h(24) = ((5 \times 24 + 2) \% 29) \% 4 = 2 \quad \} \Rightarrow \text{collisions}$

choose $\neq a, b: a=2, b=13: \Rightarrow h(S)=2$

$h(X)=3$

3) Linked hash table.

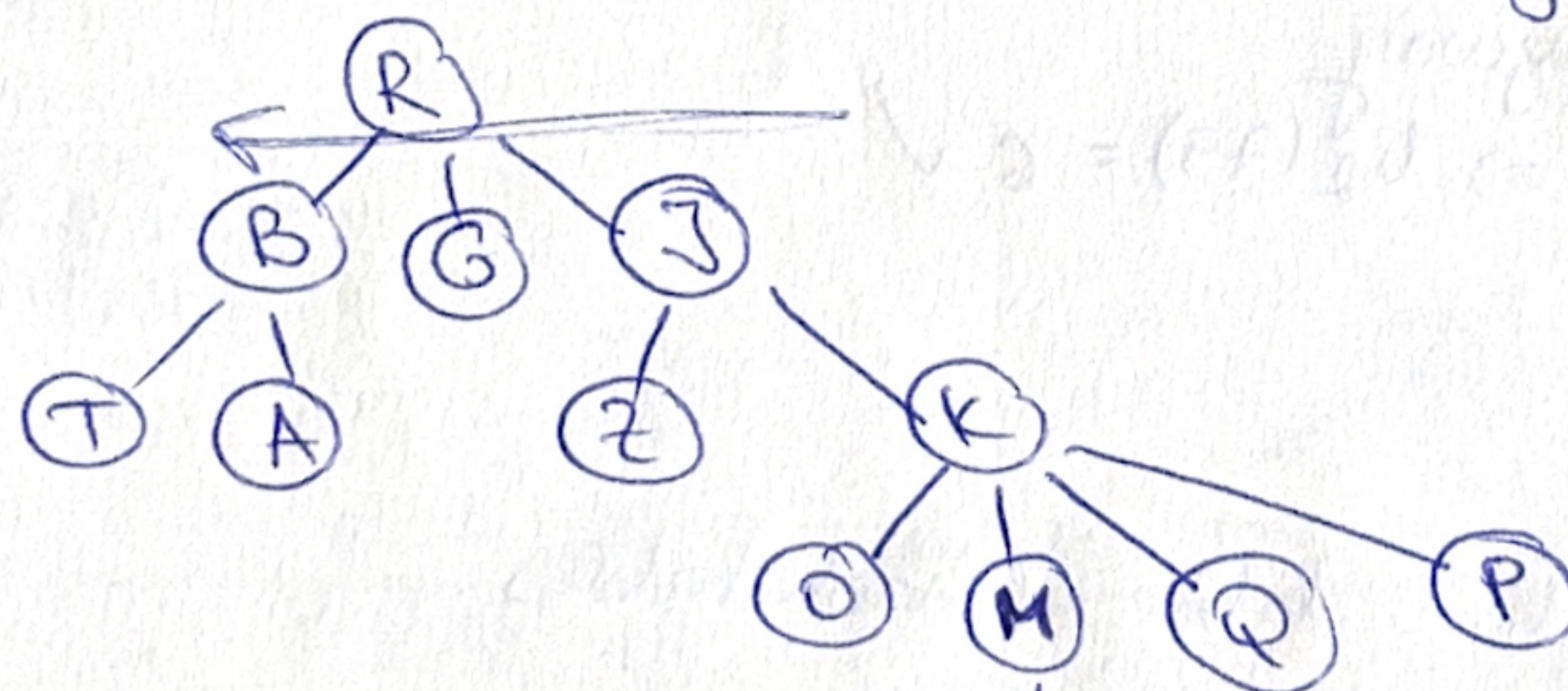
4) Trees

✓

K-ary trees. = each node has max K children.

traversals:

I. DFS $\rightarrow$ stack. (push children from right to left)



stack: ~~R~~ ~~G~~ ~~B~~ ~~A~~ ~~T~~ ~~K~~ ~~Z~~ ~~P~~ ~~Q~~ ~~Y~~

visited: R B ~~A~~ T A G J Z K O M Y Q P

II. BFS (level order) $\rightarrow$ queue. (push children from left to right)

queue: ~~R~~ ~~B~~ ~~G~~ ~~J~~ ~~T~~ ~~A~~ ~~Z~~ ~~K~~ ~~O~~ ~~M~~ ~~Q~~ ~~P~~ ~~Y~~

visited: R B G J T A Z K O M Q P Y

5) Binary trees (2-ary)

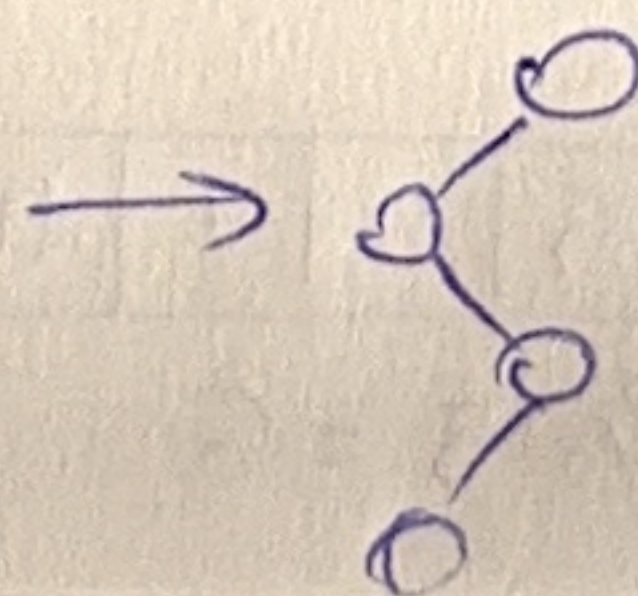
left, right child, ordered.

Full BT = nr. children 0/2

COMPLETE BT = can't add new nodes without increasing height

ALMOST COMPLETE BT = heap str.

DEGENERATE = chain (1 child)



BALANCED BT : $\text{height}(\text{left}) - \text{height}(\text{right}) \leq 1$

* n nodes (BT) $\Rightarrow n-1$ edges

• BT of height $n \Rightarrow 2^{n+1} - 1$ nodes.
(complete)

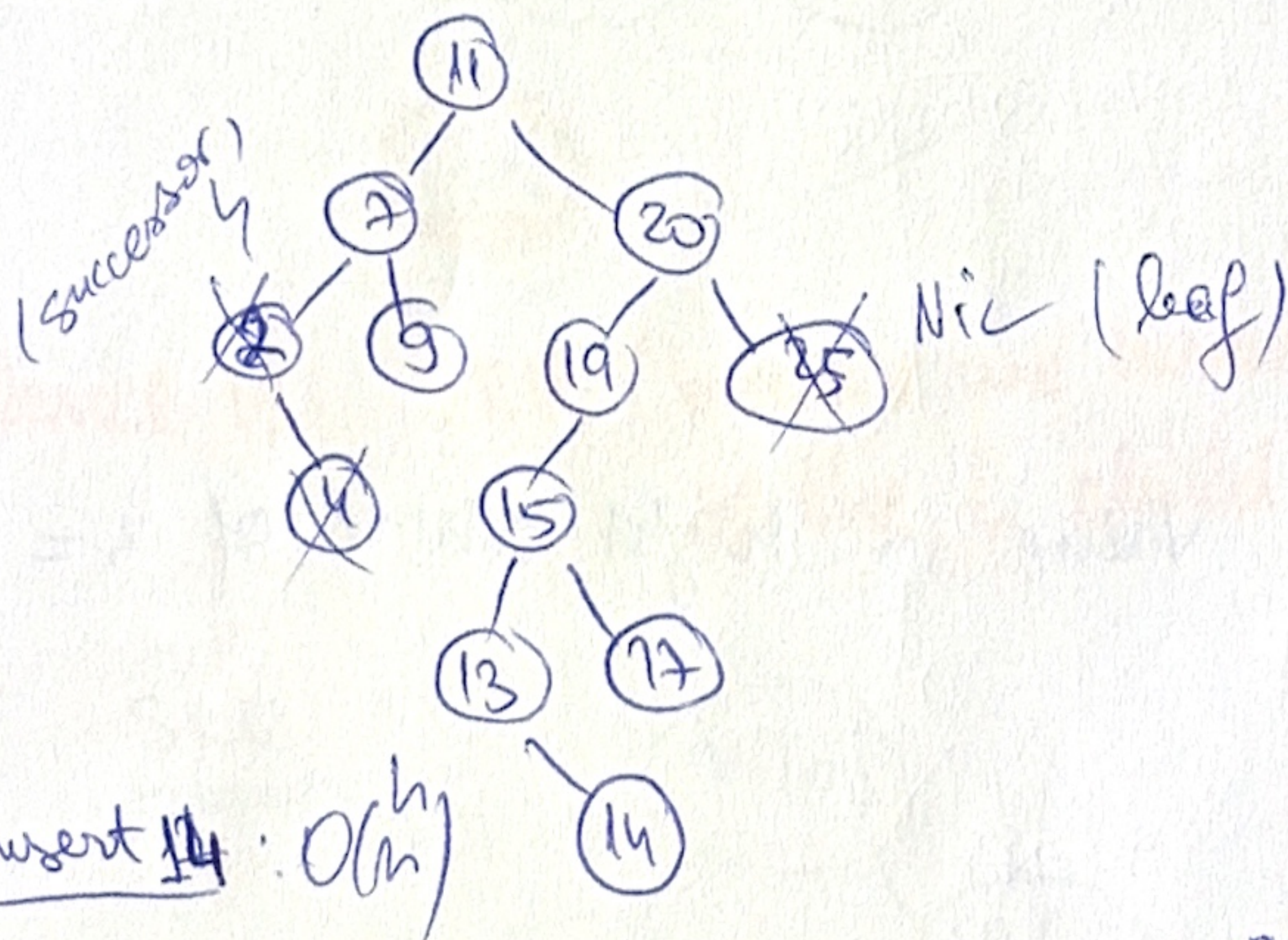
• BT of height $n \Rightarrow \text{max nr. nodes} : 2^{n+1} - 1$.
(complete)

• BT of height $n \Rightarrow \text{min. nr. nodes} : n+1$.
(degenerate)

• BT with n nodes $\Rightarrow \log_2 n \leq \text{height} \leq n-1$.

Lecture 12

1) BST.



insert 14 : $O(h)$

find min $\rightarrow$ go left as much as possible. $O(h)$

find parent $\rightarrow O(h)$

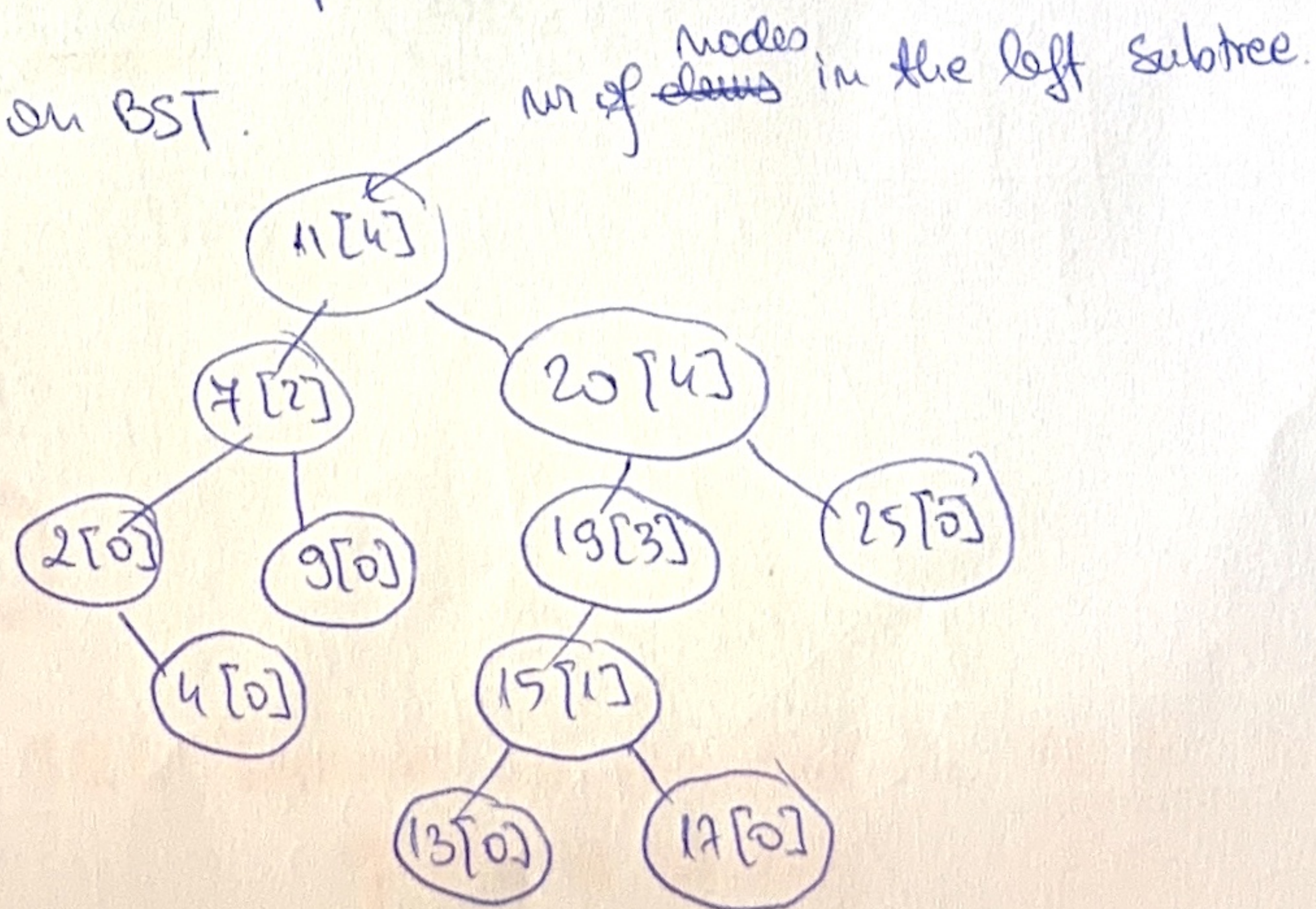
successor

remove 25

succ.

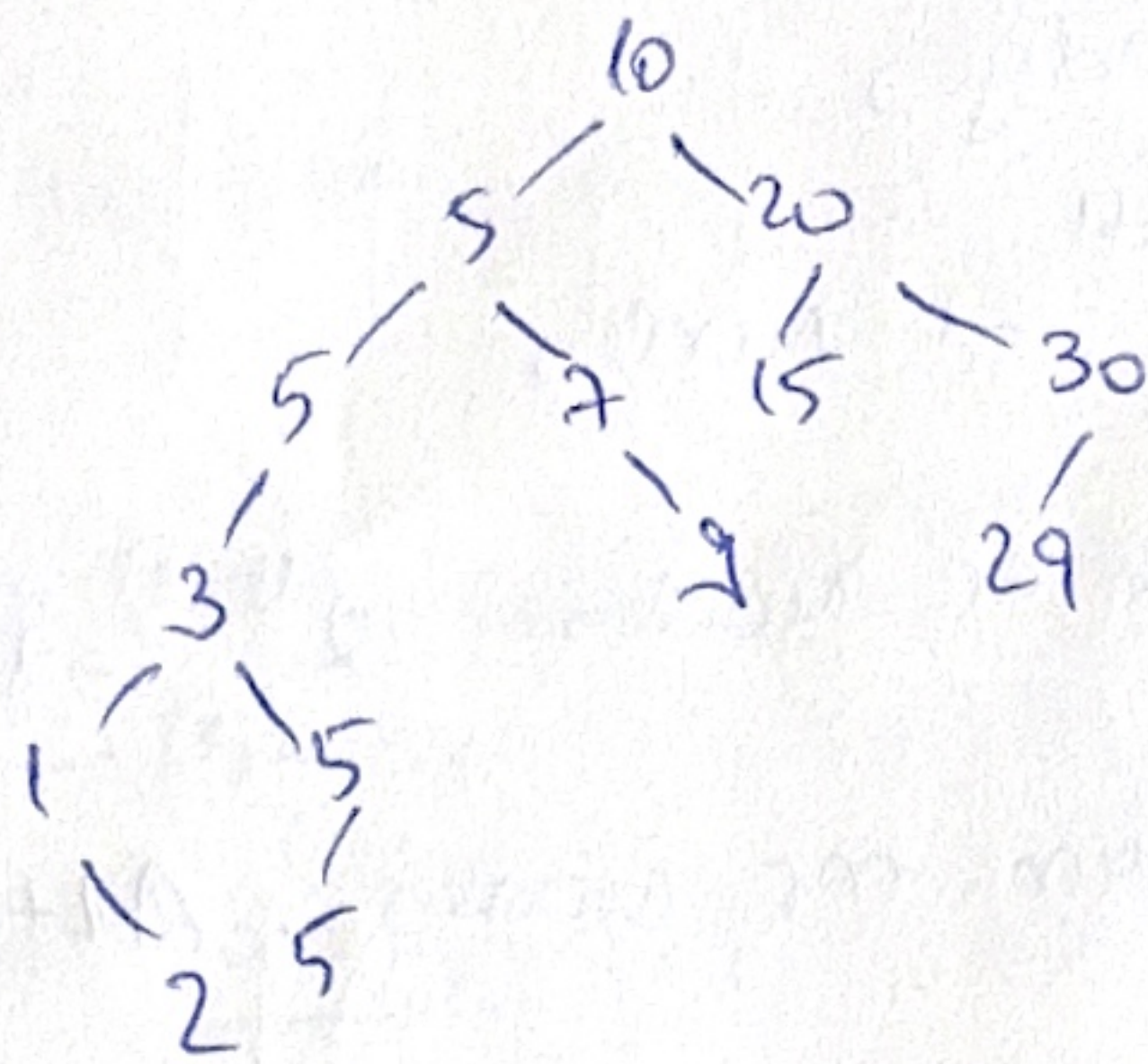
pred.

2) Sorted list on BST.



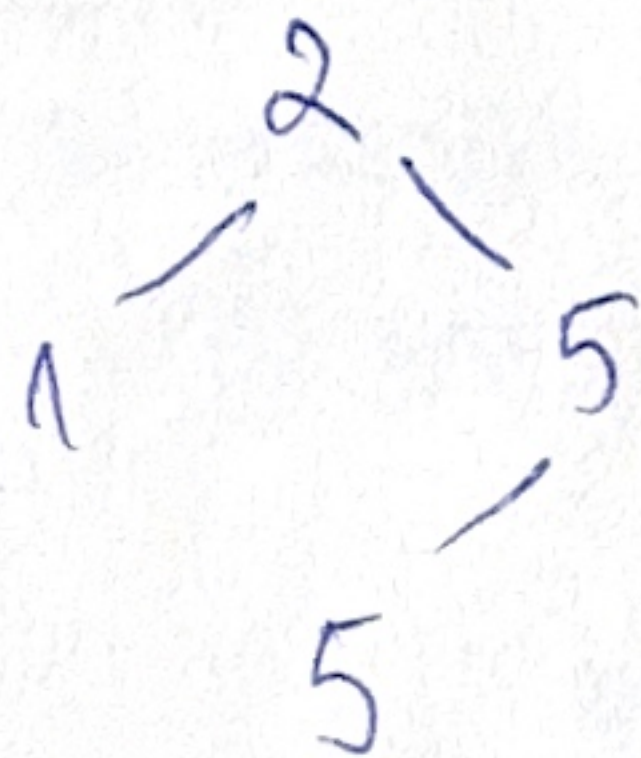
BST: $\leq$ (with duplicates)!

$\leq \Rightarrow$ left
 $> \Rightarrow$ right

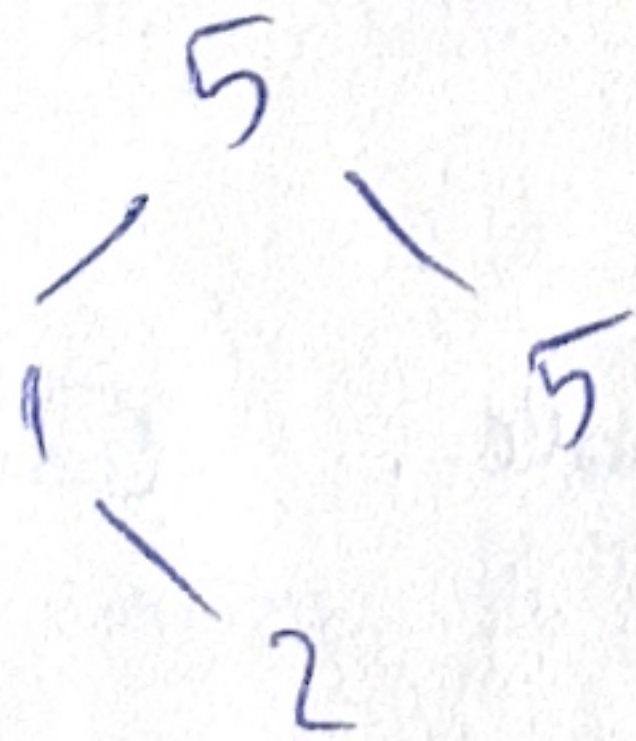


remove 3:

I: predecessor: max value in the left subtree of 3 = 2
 $\Rightarrow$ replace with 2.

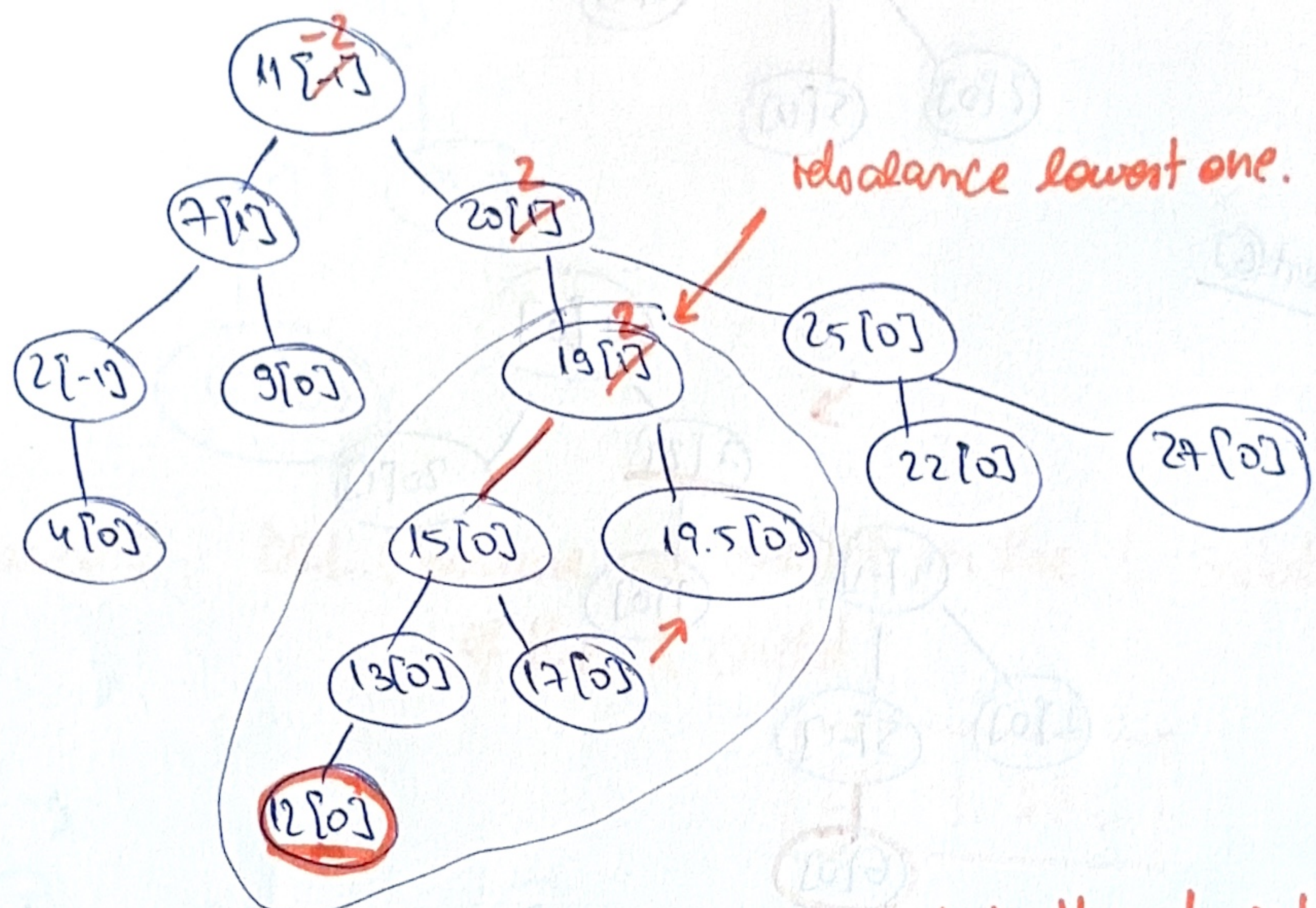


II: successor: min value in right subtree of 3 = 5



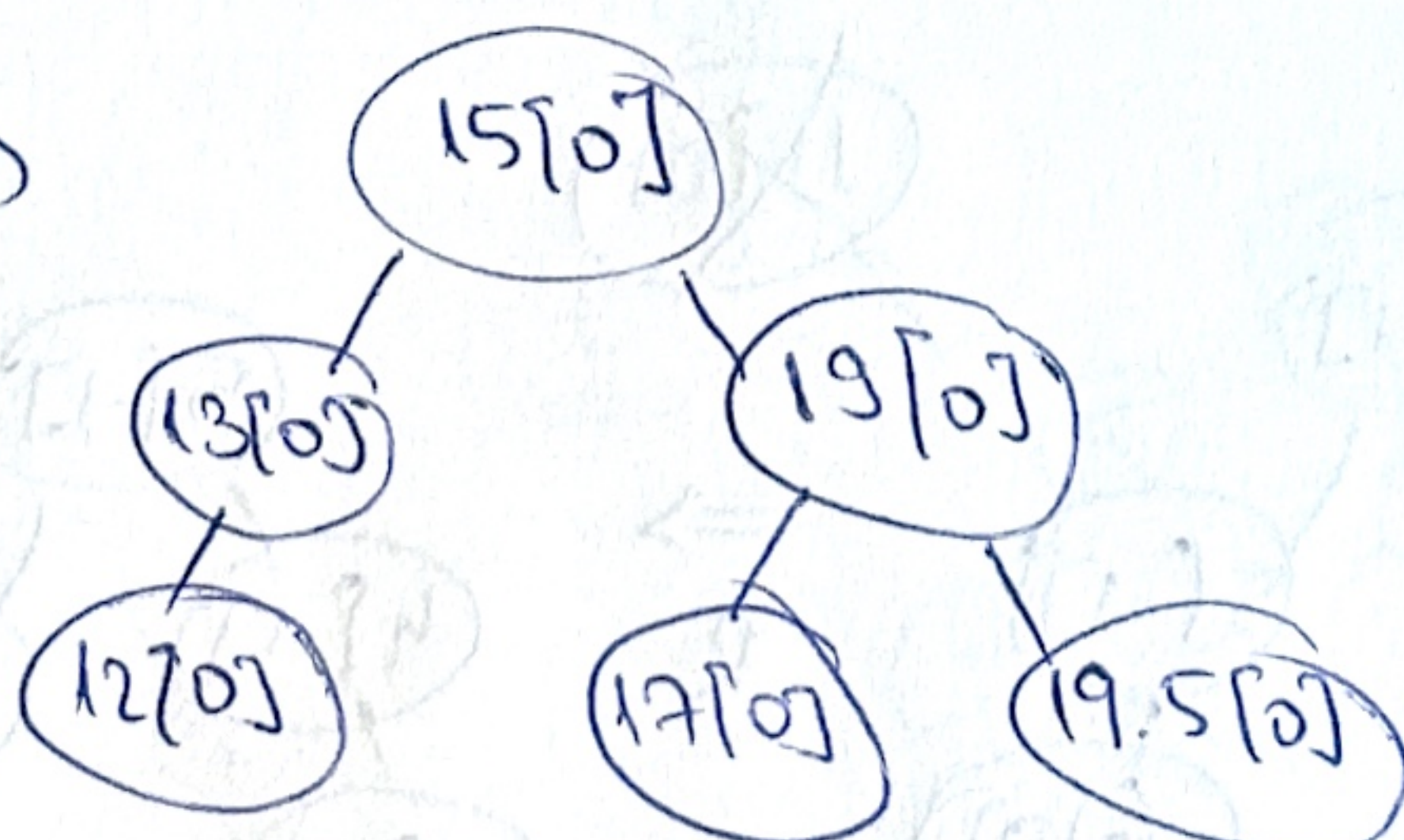
- 1) B.T Iterator
- 2) AVL Trees Huffman
- 3) AVL Trees

insert 12

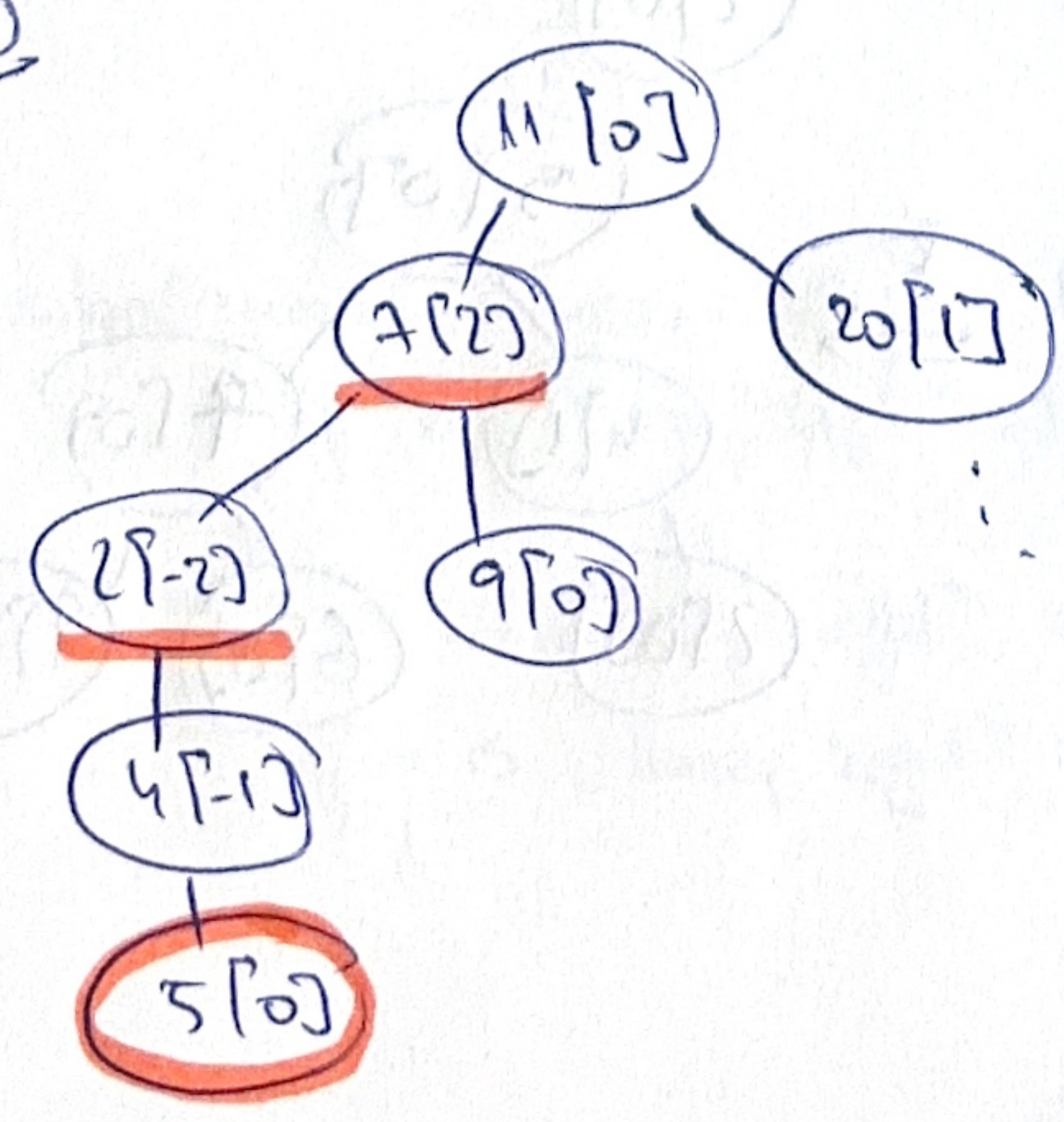


rebalance 19: SRR (because we inserted in the L subtree of the L child). $LL \Rightarrow R$

SRR

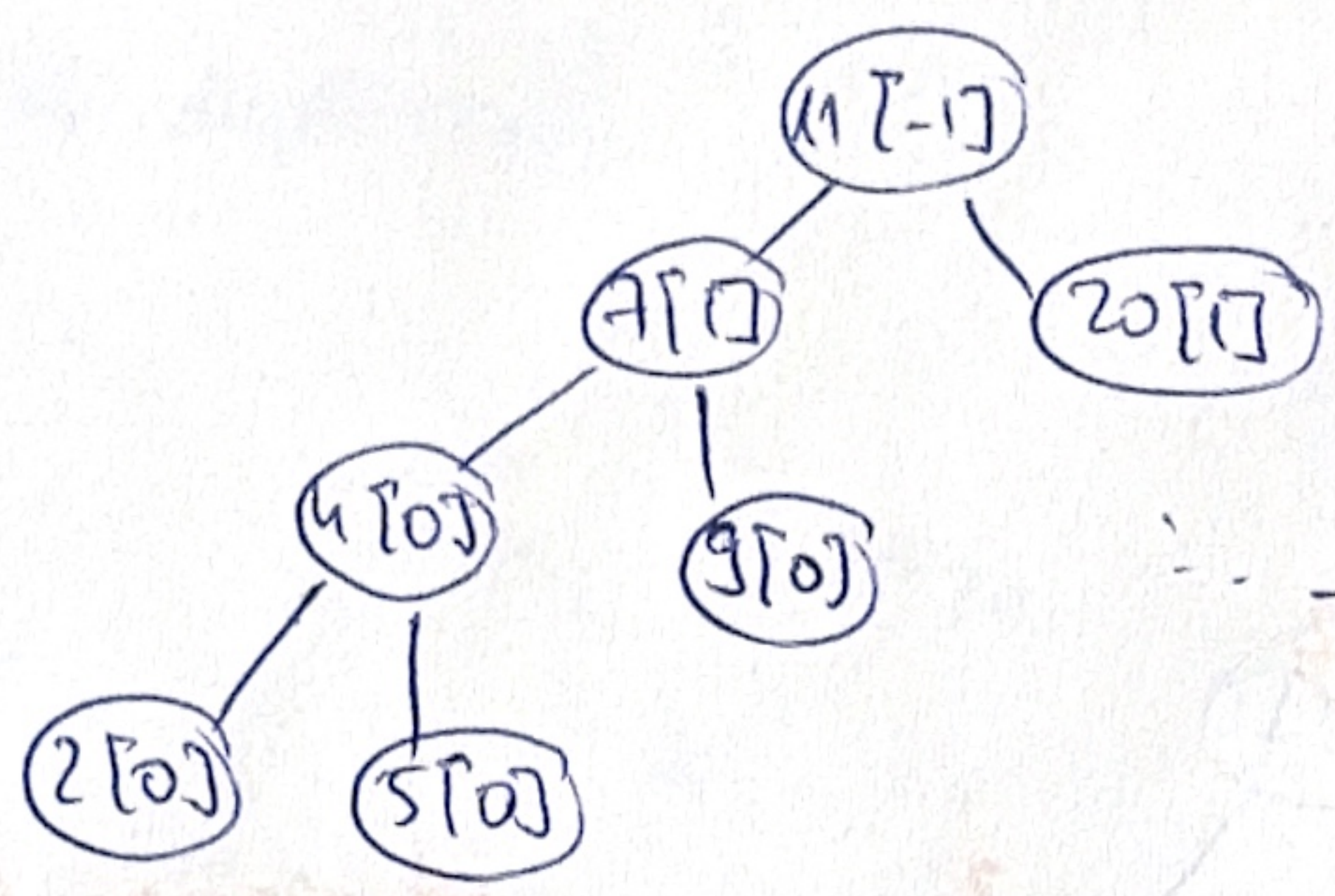


insert 5

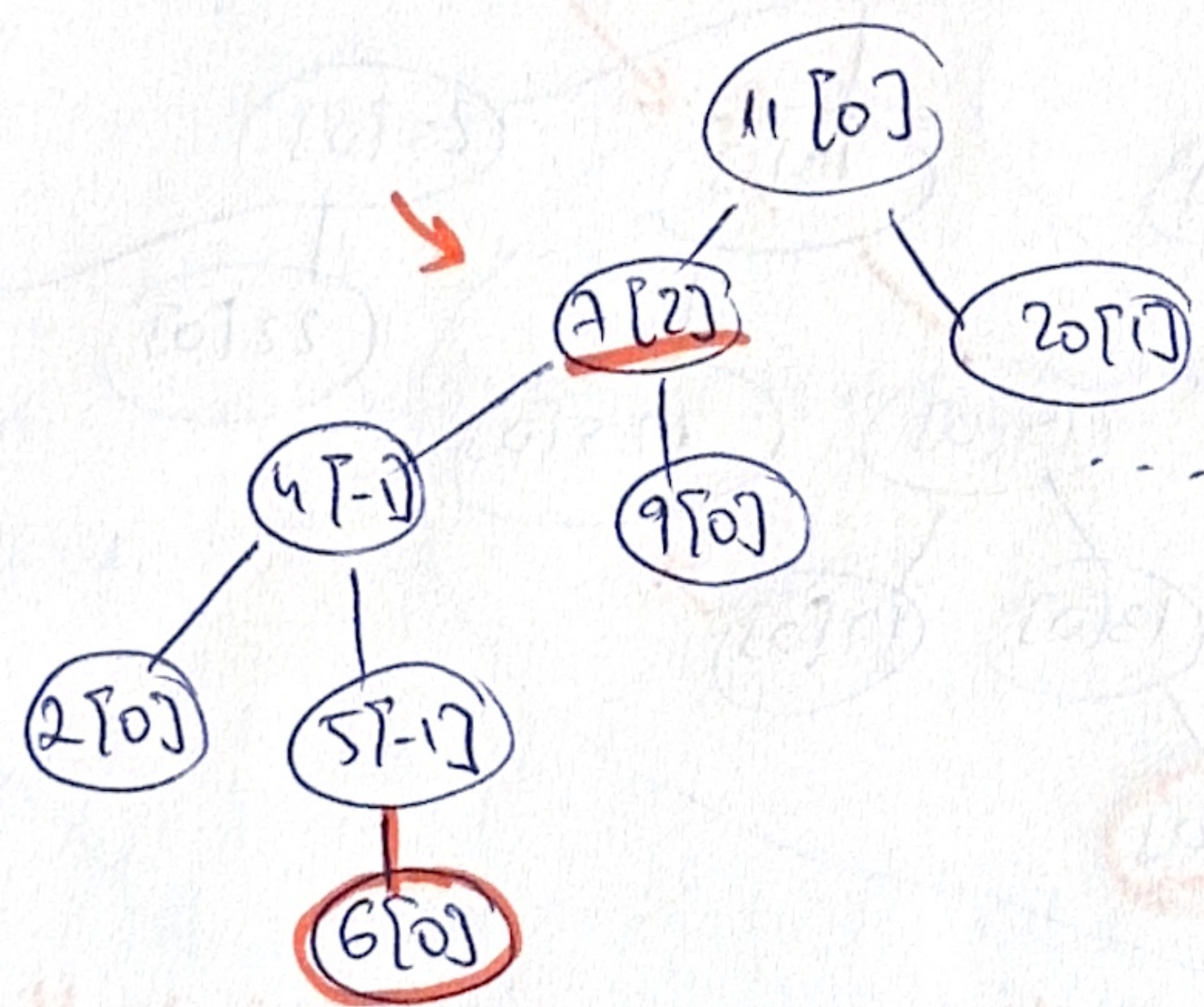


rebalance 2: SRL (because we inserted in the R subtree of the R child) $RR \Rightarrow L$

SRL →

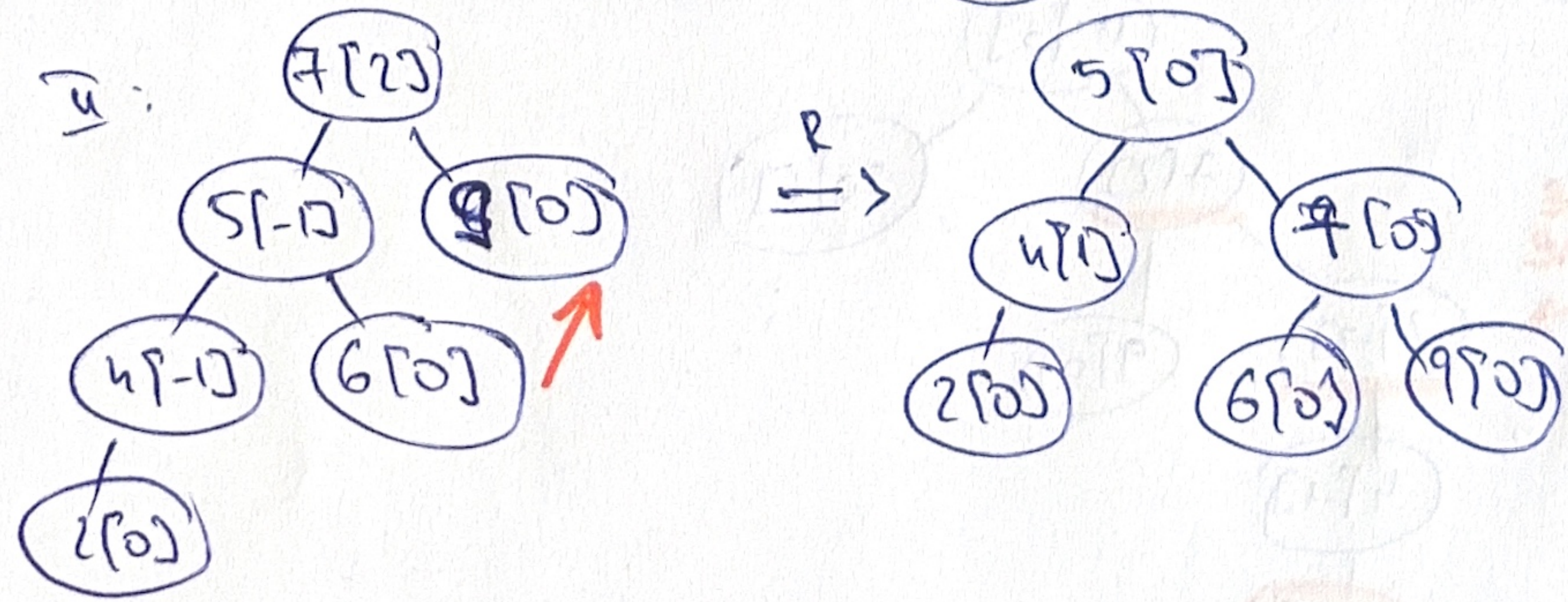
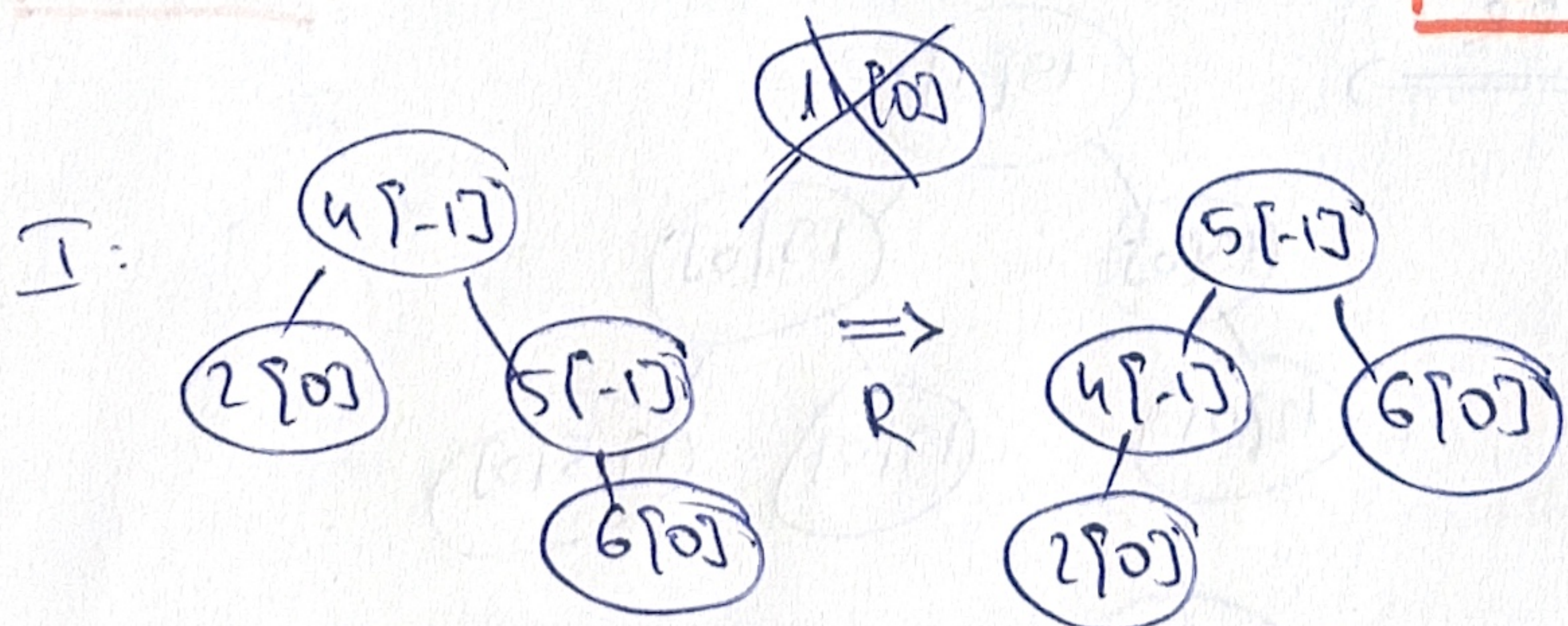


insert 6

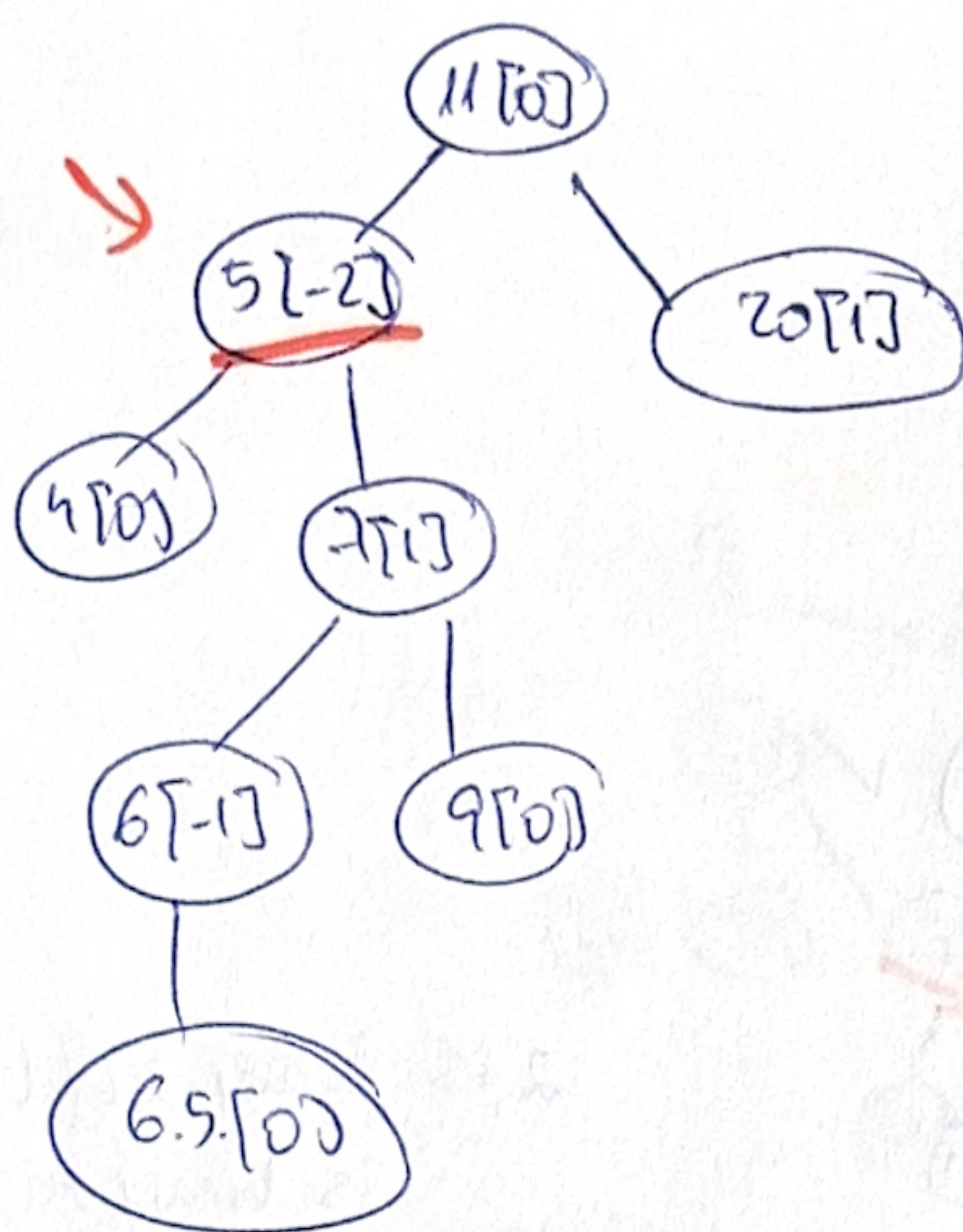


rebalance 7: DRR (because we inserted in the R subtree of the L child)

RL => DRR

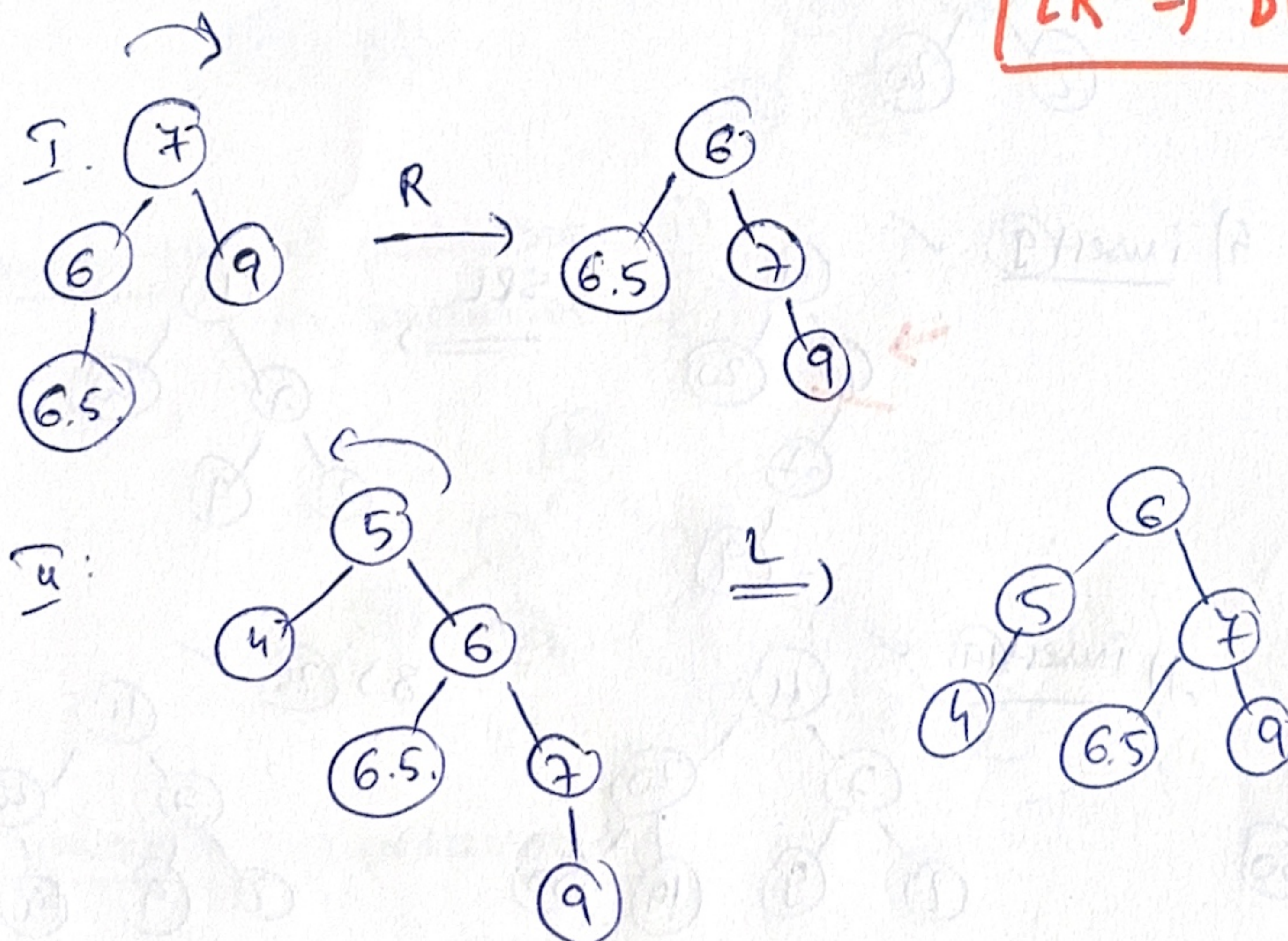


remove 2, insert 6.5



rebalance 5 : DRL (because we inserted in the L subtree of the R child)

LR $\Rightarrow$ DRL

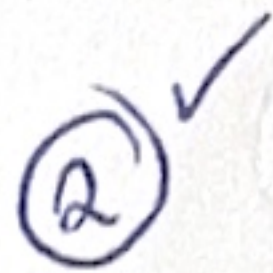


● imbalanced $\rightarrow$ "heavy-left" $\Rightarrow$ look at its left child $\Rightarrow$
if it is "heavy-left" $\Rightarrow$ single rotation, otherwise double.

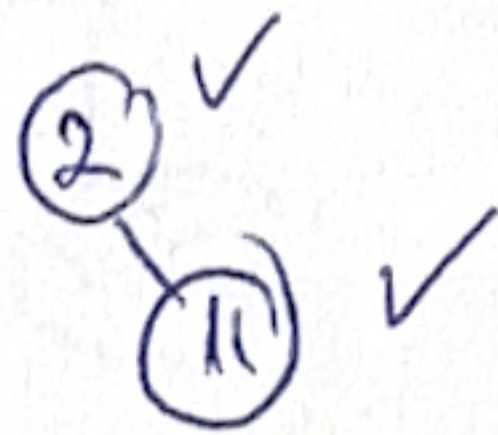
$\rightarrow$ "heavy-right" $\Rightarrow$... right $\Rightarrow$
if it is "heavy-right" $\Rightarrow$ single, otherwise double.

empty AVL.

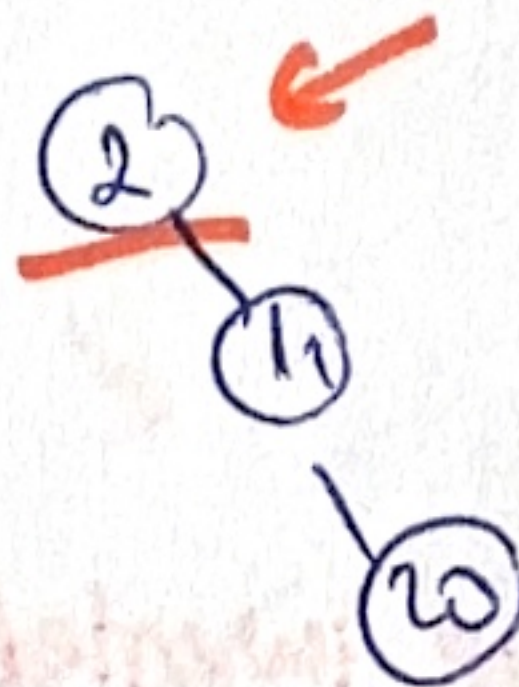
1) insert 2 : ✓



2) insert 11 ✓



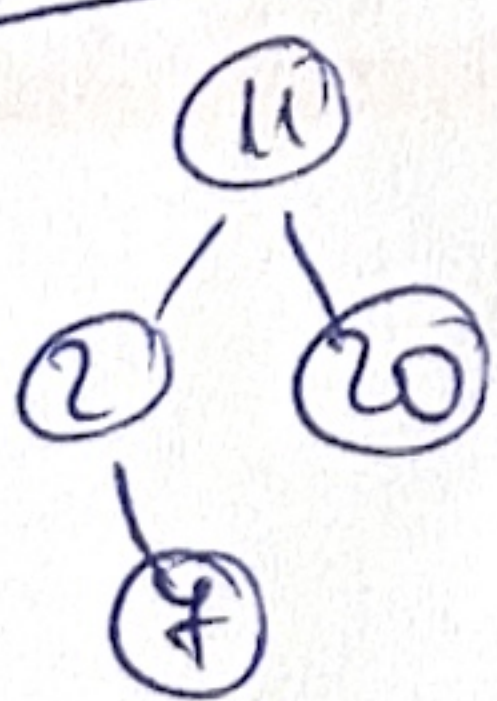
3) insert 20



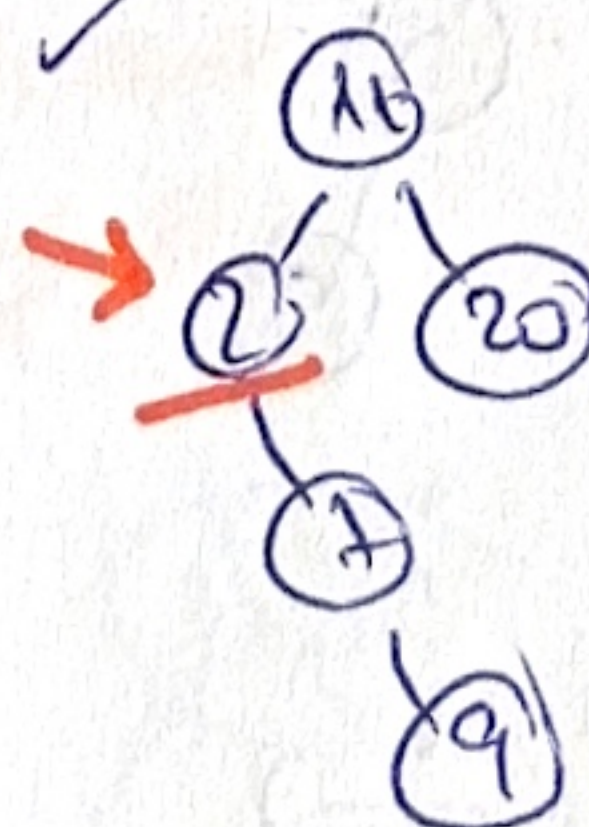
2 is heavy-right, its right child is heavy right $\Rightarrow$ SRL



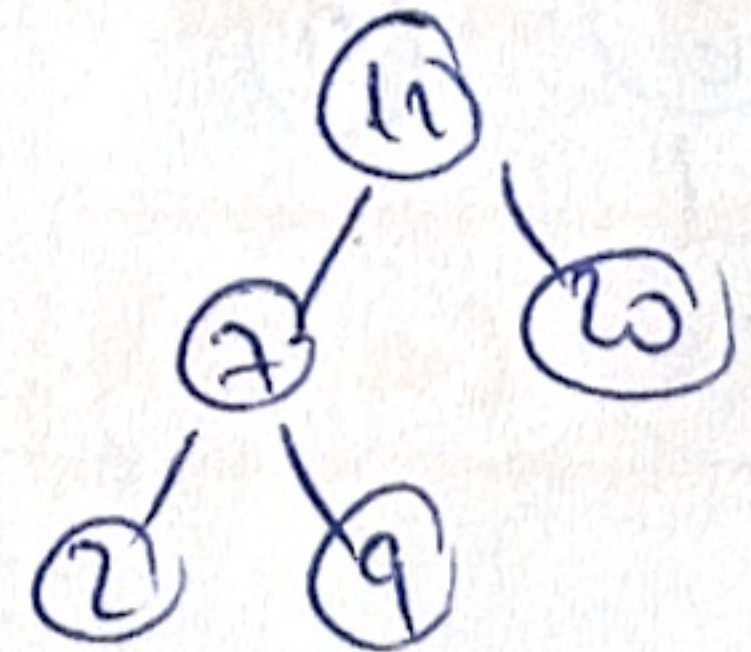
4) insert 7



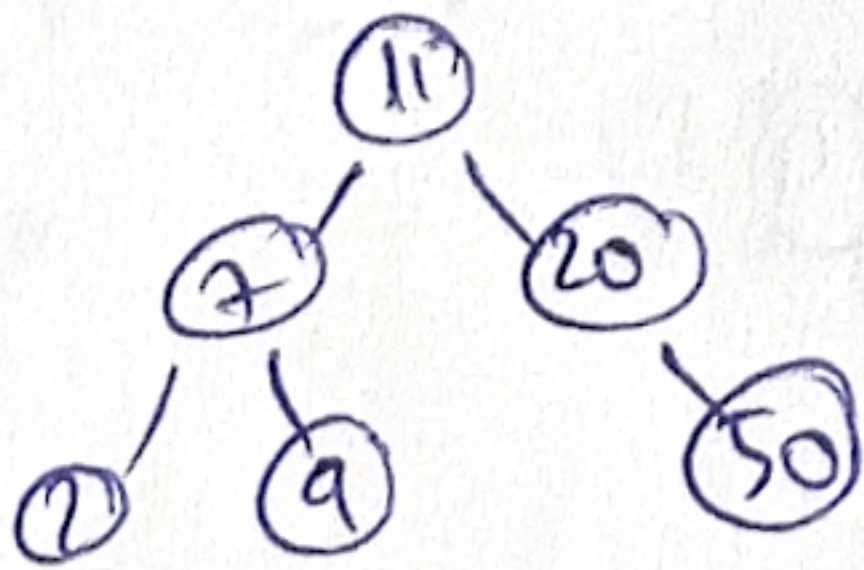
5) insert 9 ✓



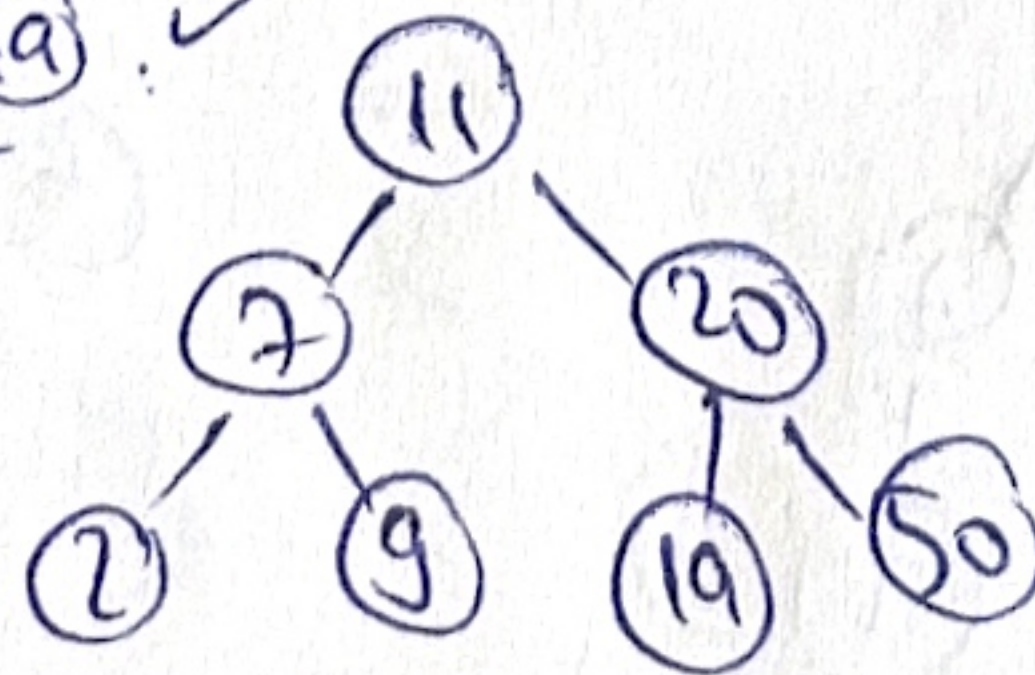
$\xRightarrow{\text{SRL}}$



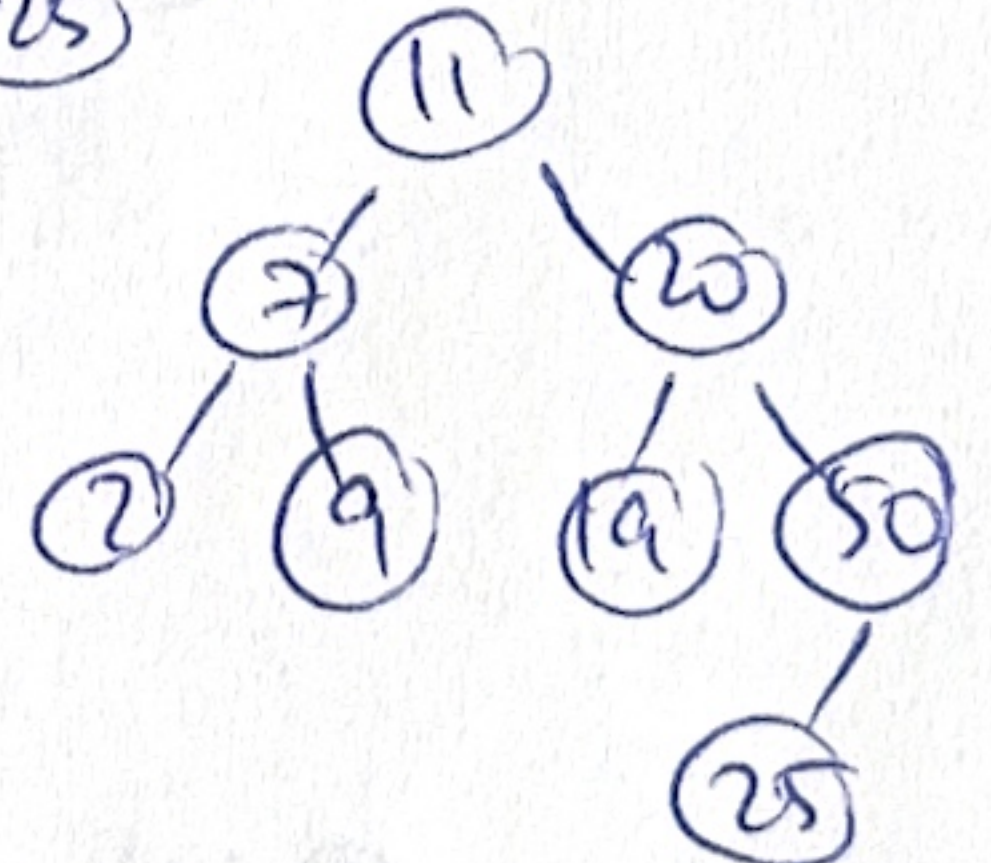
6) insert 50 ✓



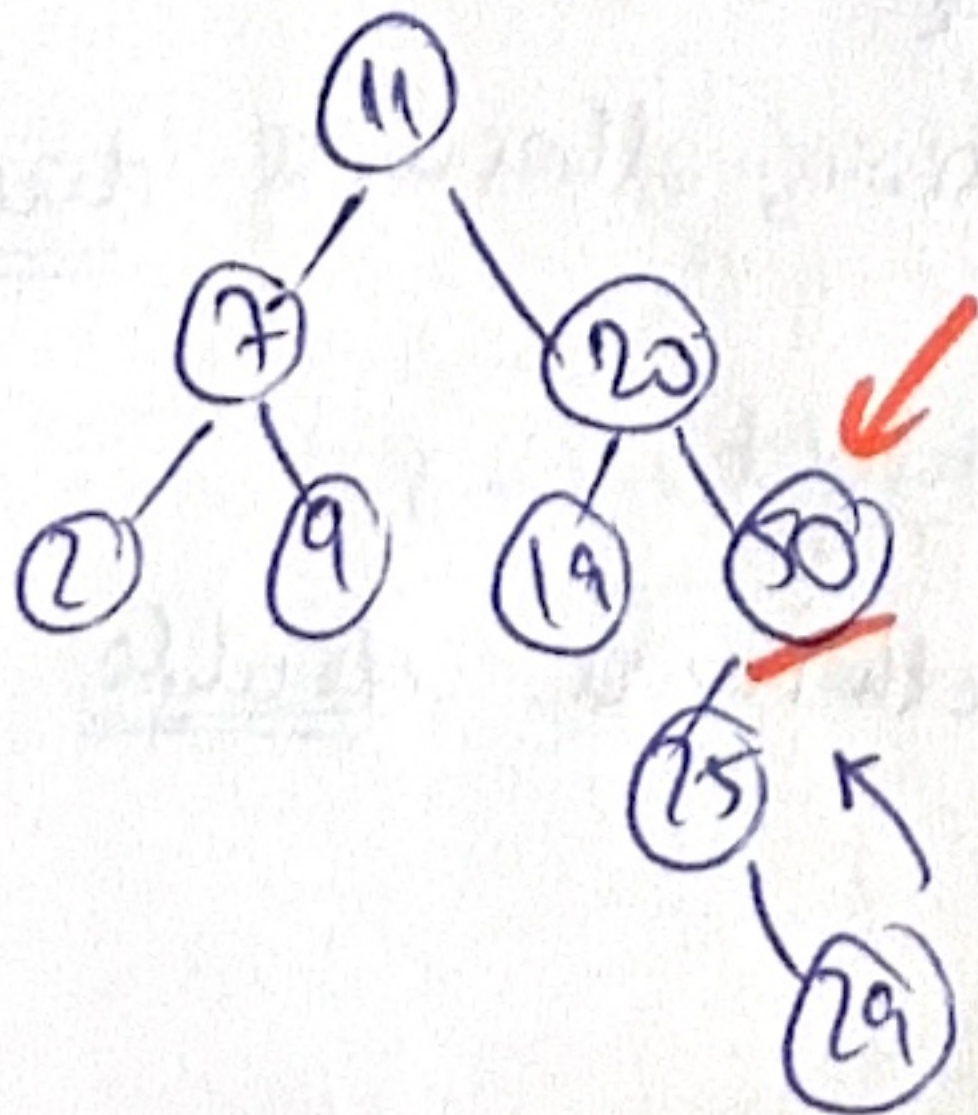
7) insert 19 ✓



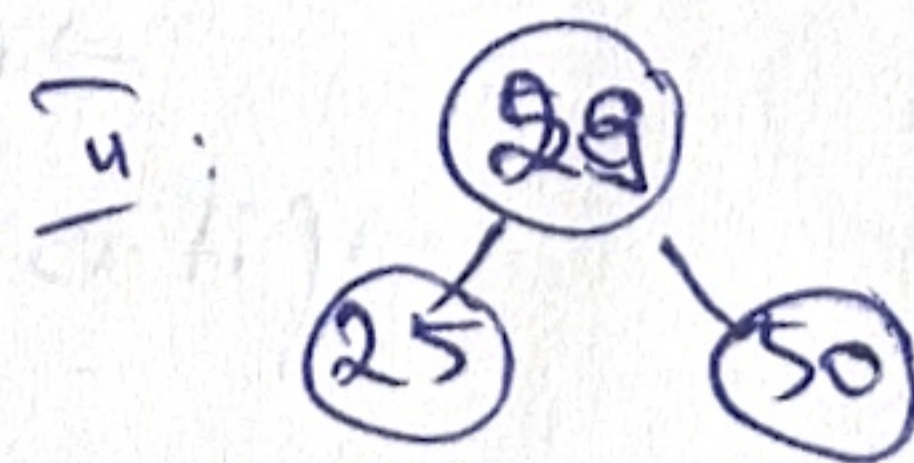
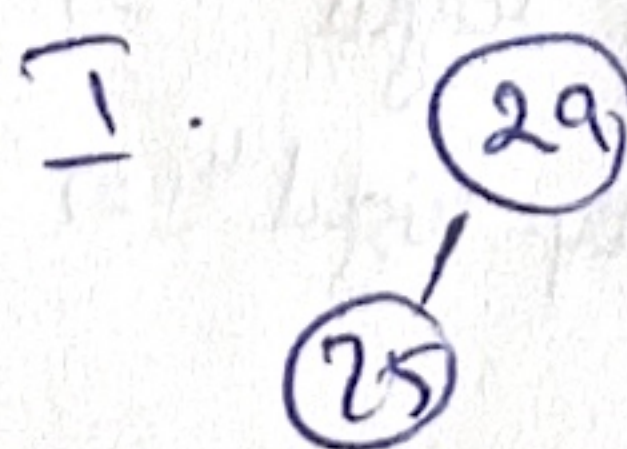
8) 25 ✓



9) ins 29

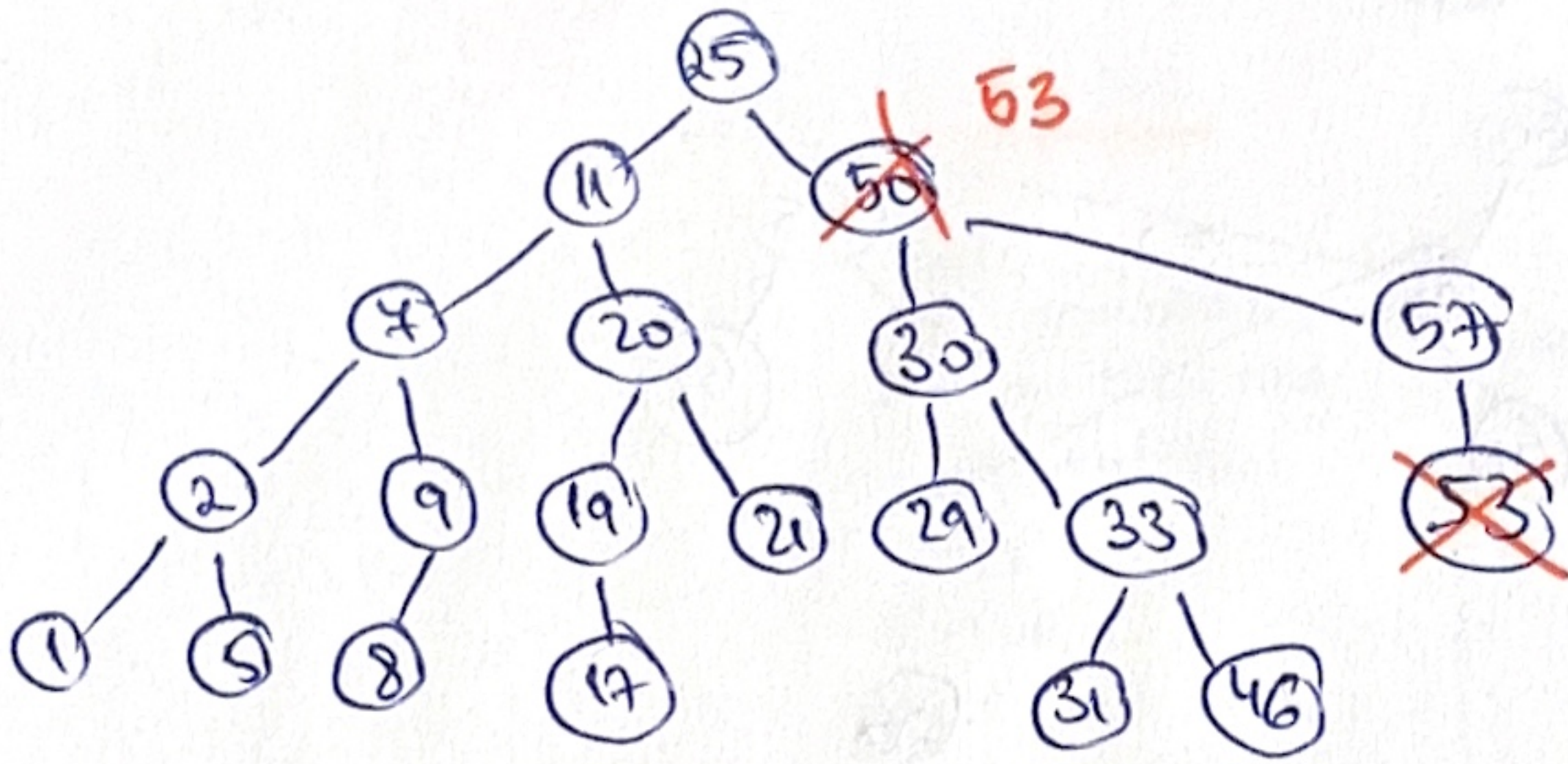


30 is heavy-left, its left child is heavy right $\Rightarrow$ DRR

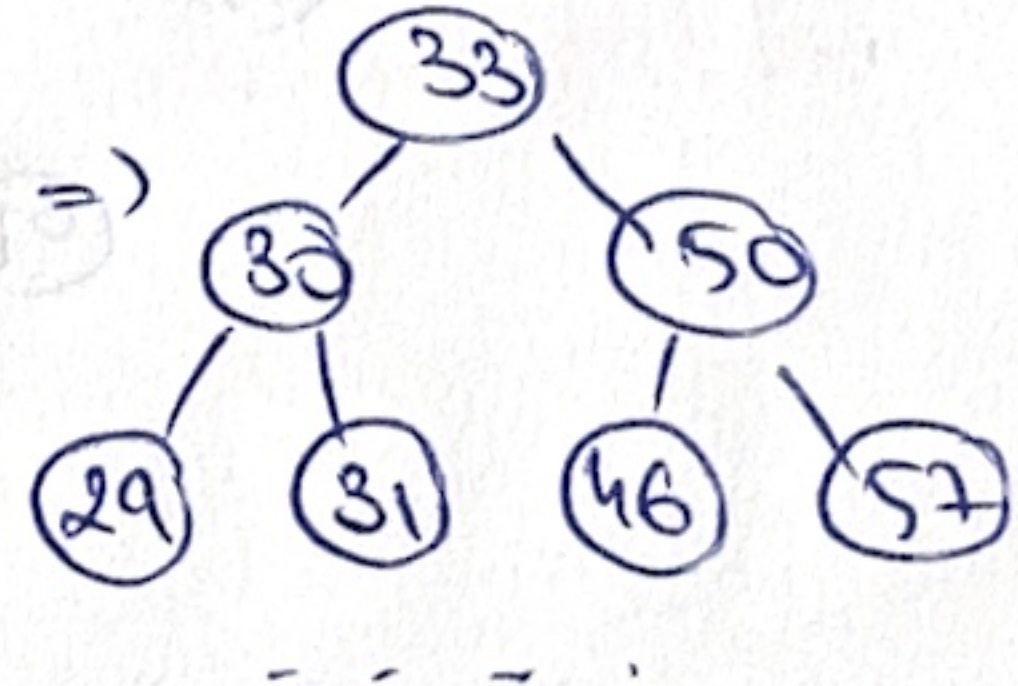
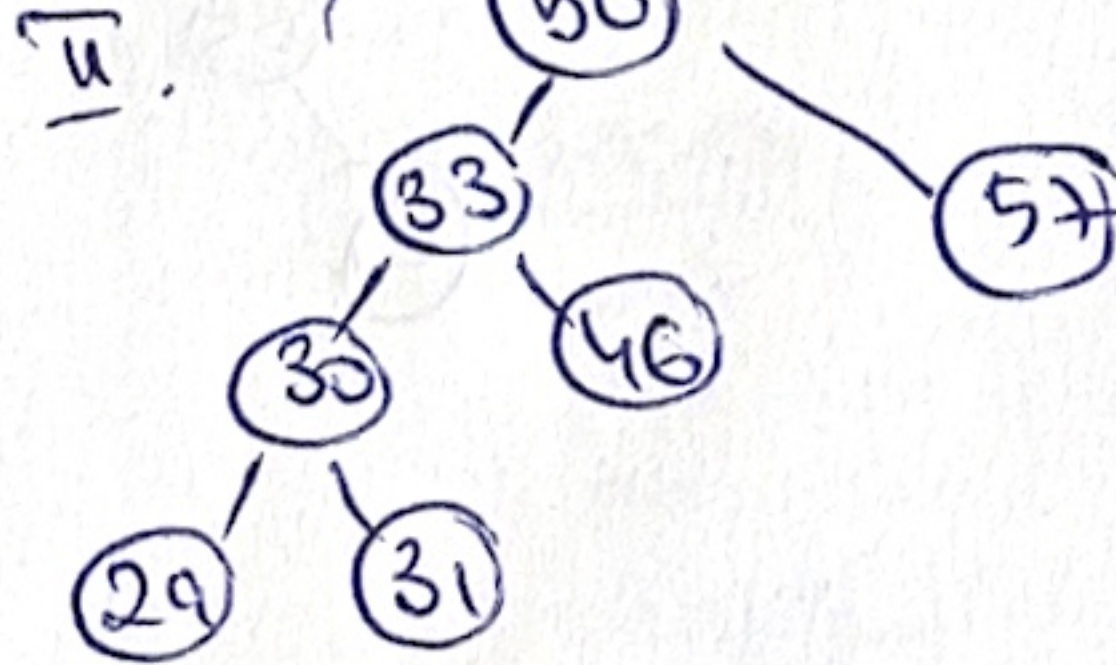
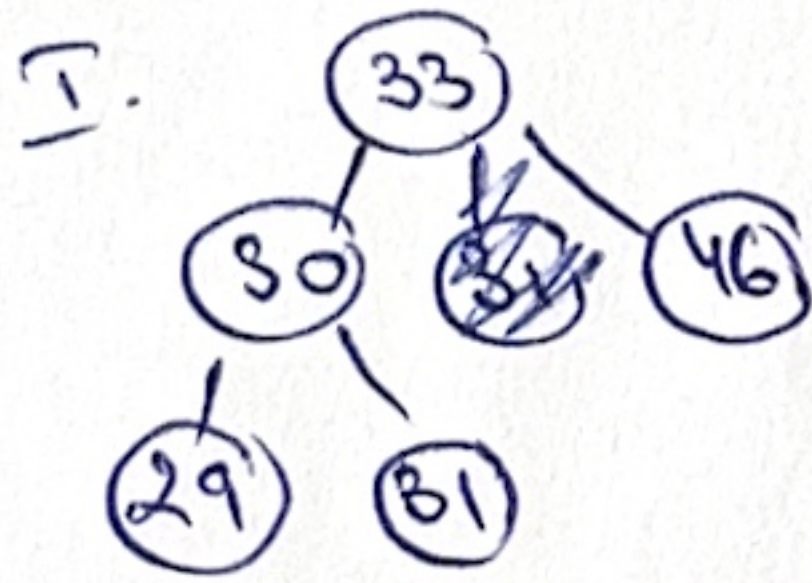


Lecture 14

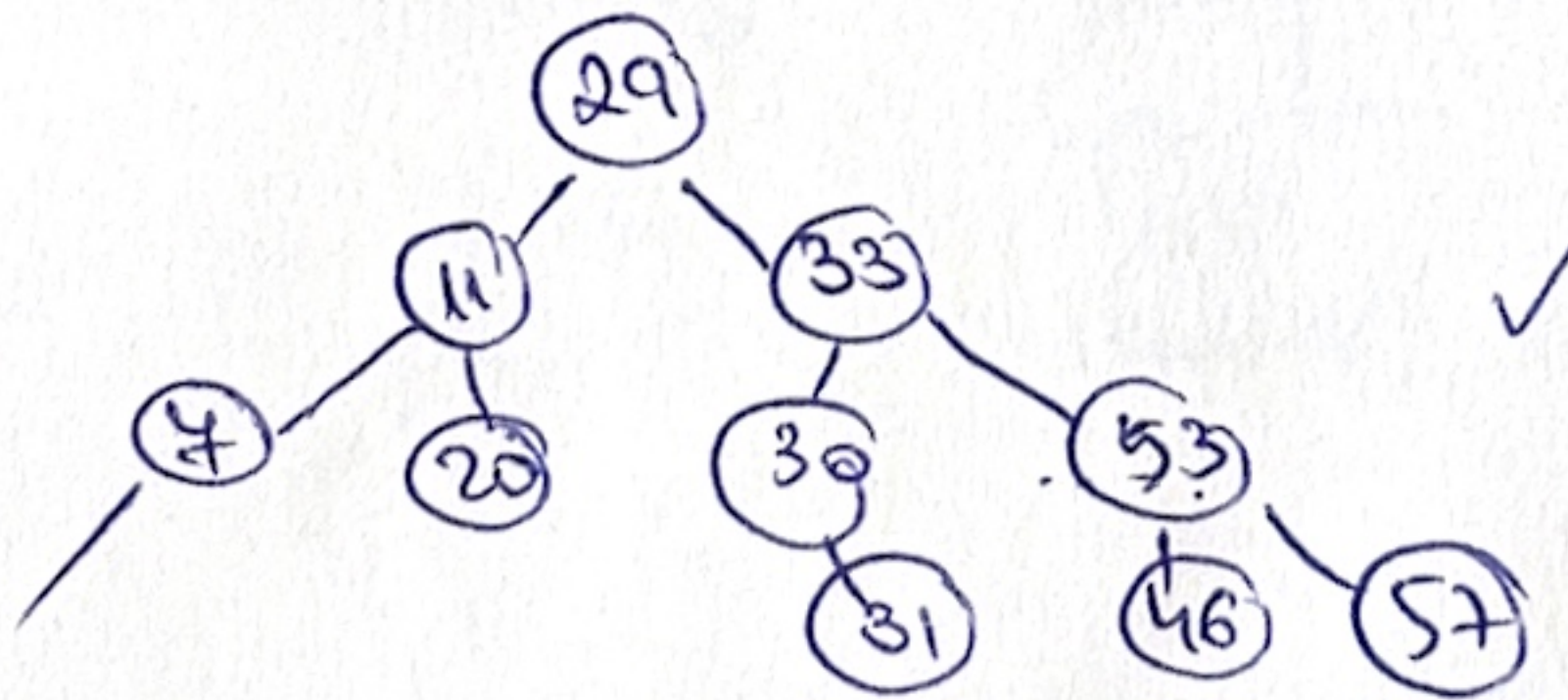
1) Remove from AVL tree.



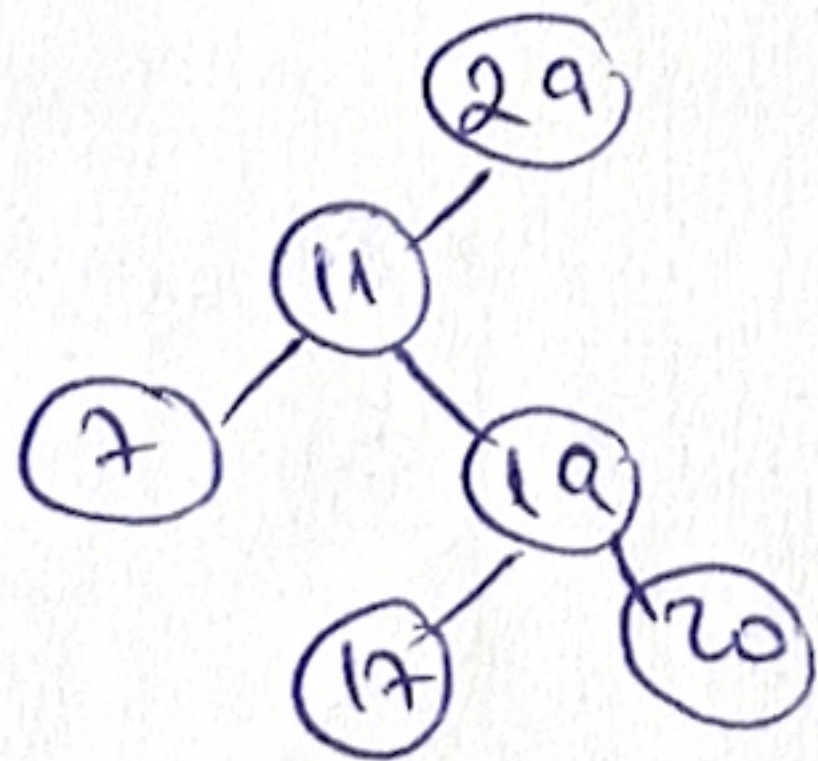
remove 50 $\Rightarrow$ (successor) rebalance 53 $\Rightarrow$ DRR $\Rightarrow$



remove 25 $\Rightarrow$ predecessor: 21
 $\Rightarrow$ successor: 29

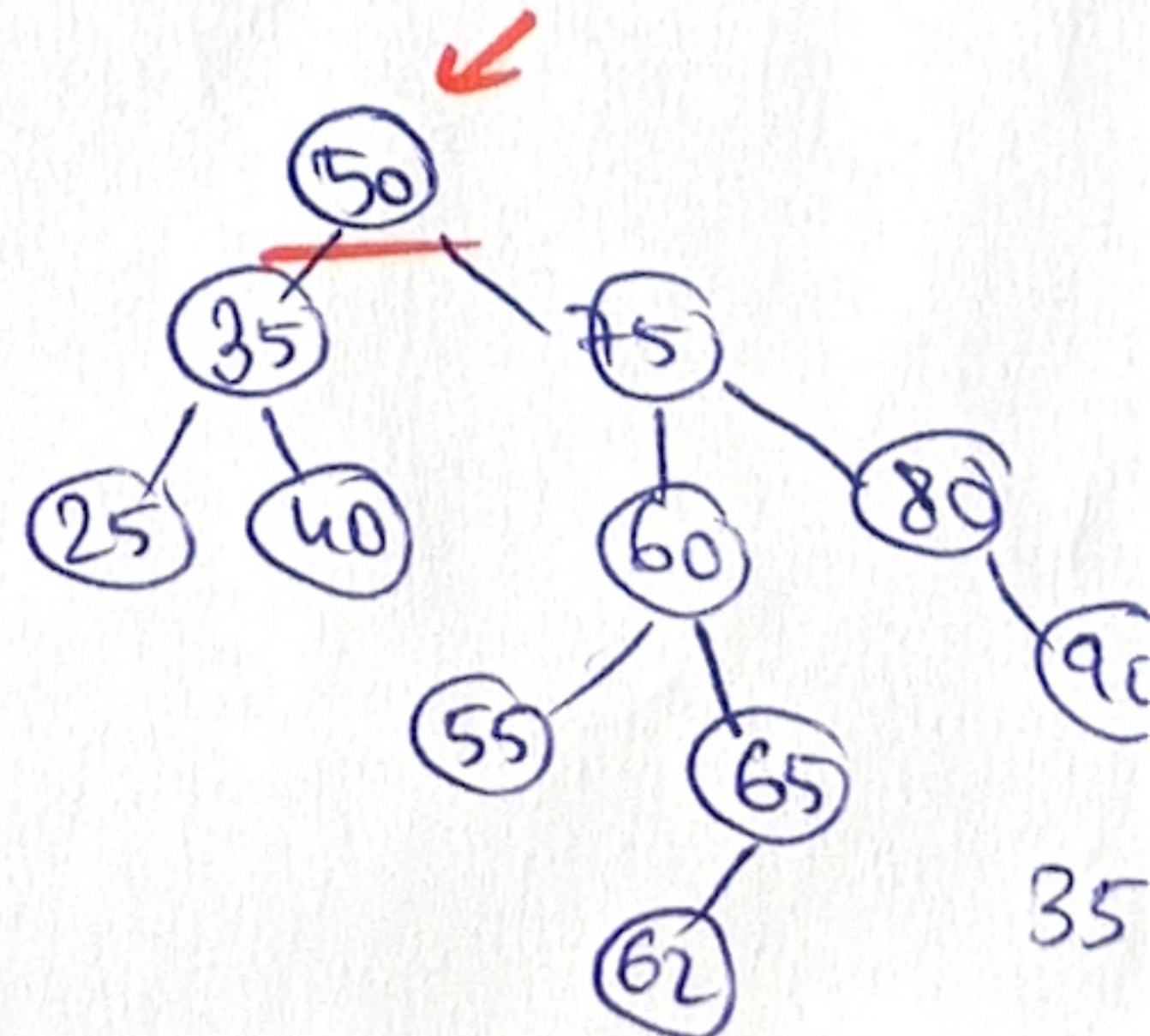
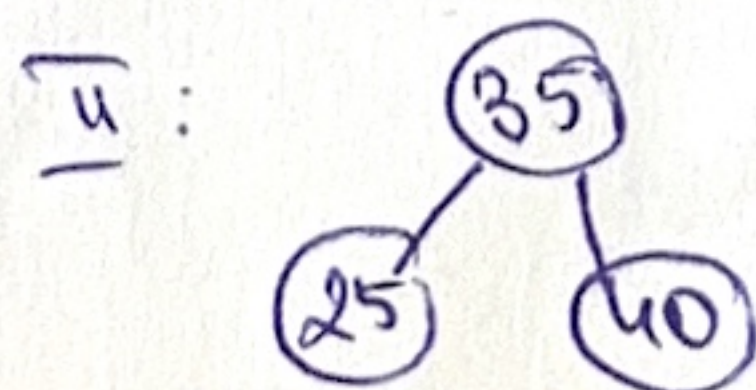
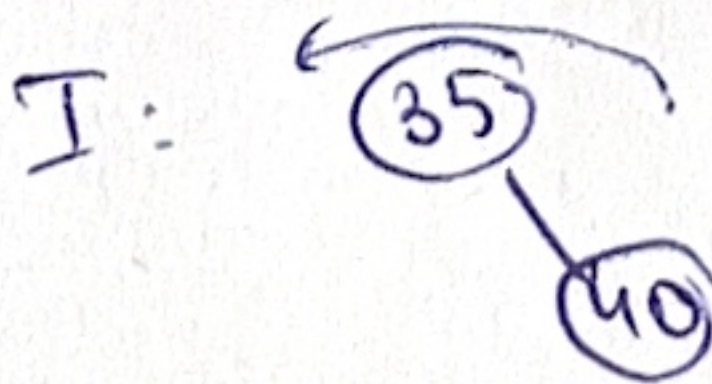


remove 20 $\Rightarrow$ predecessor: 19

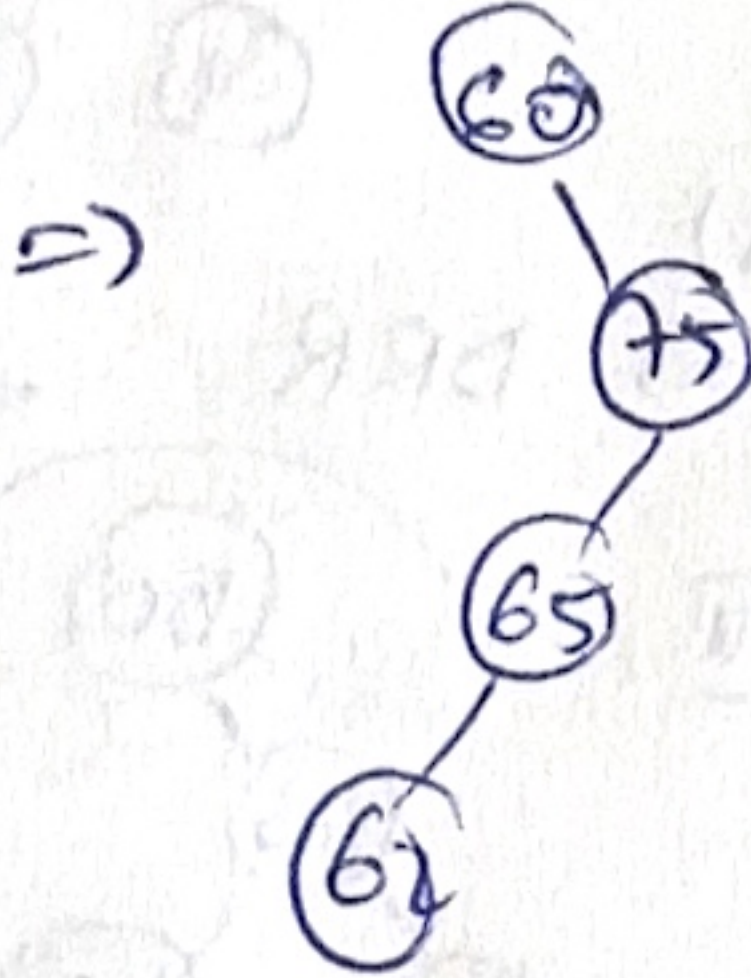
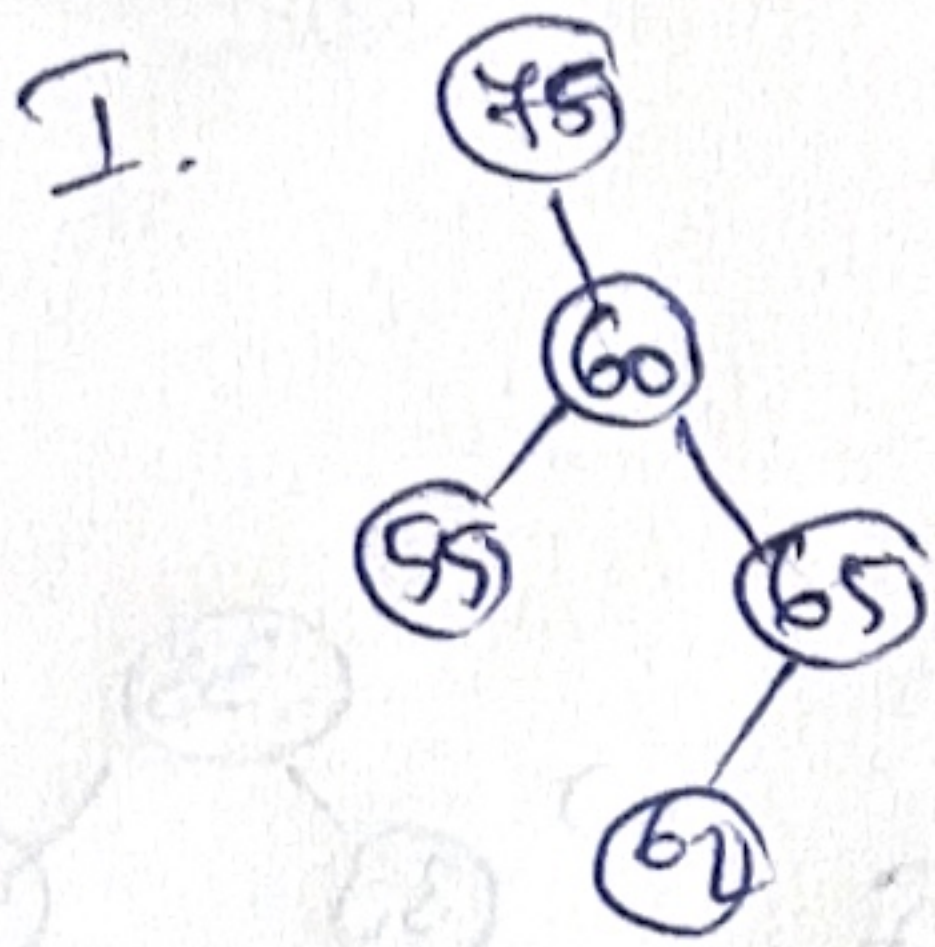
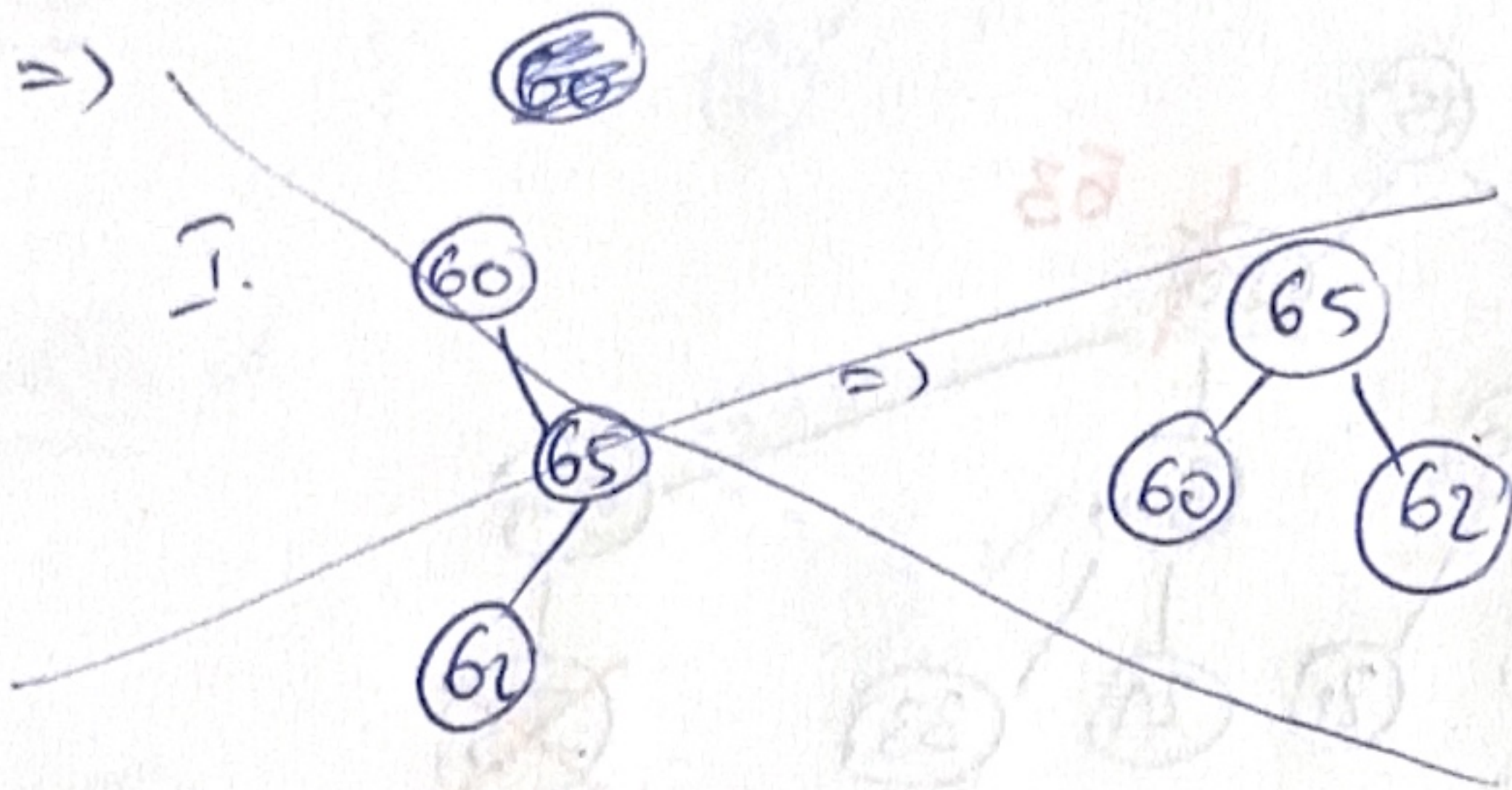


remove 15:

rebalance 25: DRL



rebalance 50: **DRL** (50 heavy right; its right child is heavy left)



rebalance 50: **DRL**

